
Table of Contents

Introductions

Introduction	1.1
Introductions	1.2

Getting Started

Getting Ready for Class	2.1
Getting Started	2.2

Project 1: Caption This

Branching with Git	3.1
Local Git Configs	3.2

GitHub Training Manuals

Welcome to the GitHub Training Manual repo. In this repo, you will find the content for all of the materials used by GitHub trainers during our official training courses.

How To Generate the Manuals

First, ensure that you've installed all the required dependencies by running:

```
script/setup
```

This will also generate student facilitator copies of the manual in Markdown, HTML, and PDF.

To serve the manuals as a Jekyll web page, run:

```
script/server
```

To generate a manual other than *GitHub for Developers* run:

```
`script/generate-manual`
```

where `<manual-name>` is one of the `*.md` files in this project's root directory.

Welcome to GitHub for Developers

Today you will embark on an exciting new adventure: learning how to use Git and GitHub.

As we move through today's materials, please keep in mind: this class is for you! Be sure to follow along, try the activities, and ask lots of questions!

License

The prose, course text, slide layouts, class outlines, diagrams, HTML, CSS, and Markdown code in the set of educational materials located in this repository are licensed as [CC BY 4.0](https://creativecommons.org/licenses/by/4.0/). The Octocat, GitHub logo and other already-copyrighted and already-reserved trademarks and images are not covered by this license.

For more information, visit: <http://creativecommons.org/licenses/by/4.0/>

Getting Ready for Class

While you are waiting for class to begin, please take a few minutes to set up your local work environment.

Step 1: Set Up Your GitHub.com Account

For this class, we will use a public account on GitHub.com. We do this for a few reasons:

- We don't want you to "practice" in repositories that contain real code.
- We are going to break some things so we can teach you how to fix them.
(therefore, refer to the bullet above)

If you already have a github.com account you can skip this step. Otherwise, you can set up your free account by following these steps:

1. Access GitHub.com and click Sign up.
2. Choose the free account. 1.You will receive a verification email at the address provided.
3. Click the link to complete the verification process.

Step 2: Install Git

Git is an open source version control application. You will need Git installed for this class.

You may already have Git installed so let's check! Open Terminal if you are on a Mac, or PowerShell if you are on a Windows machine, and type:

```
$ git --version
```

You should see something like this:

```
$ git --version  
git version 2.11.0
```

Anything over 2.0 will work for this class!

Downloading and Installing Git

If you don't already have Git installed, you can download Git at www.git-scm.com.

If you need additional assistance installing Git, you can find more information in the ProGit chapter on installing Git: [http://git-scm.com/book/en/v2/Getting-Started-](http://git-scm.com/book/en/v2/Getting-Started-Installing-Git)

[Installing-Git](http://git-scm.com/book/en/v2/Getting-Started-Installing-Git) .

Where is Your Shell?

Now is a good time to create a shortcut to the command line application you will want to use with Git:

- If you are working on Windows, you can use `Git Bash` which is installed with the Git package or `Git Shell` which is installed with GitHub Desktop.
- If you are working on a Mac or other Unix based system, you can use the terminal application.

Go ahead and open your command line application now!

Step 3: Set Up Your Text Editor

For this class, we will use a basic text editor to interact with our code. Let's make sure you have one installed and ready to work from the command line.

Pick Your Editor

You can use almost any text editor, but we have the best success with the following:

- GitPad
- [Atom](#)
- Vi or Vim
- Sublime
- Notepad or Notepad++

If you do not already have a text editor installed, go ahead and download and install one of the above editors now!

Your Editor on the Command Line

After you have installed an editor, confirm you can open it from the command line.

If you are working on a Mac, you will need to Install Shell Commands from the Atom menu, this happens as part of the installation process for Windows. {:.note}

If installed properly, the following command will open the Atom text editor:

```
$ atom .
```

Exploring

Congratulations! You should now have a working version of Git and a text editor on your system. If you still have some time before class begins, here are some interesting resources you can check out:

- github.com/explore Explore is a showcase of interesting projects in the GitHub Universe. See something you want to re-visit? Star the repository to make it easier to find later.
- services.github.com/on-demand Our On Demand Training courses are GitHub's open source training materials. The site contains additional resources you may find helpful when reviewing what you have learned in class! You can even make contributions to the materials or open issues if you would like us to explain something in greater detail. Find the open source repository here:

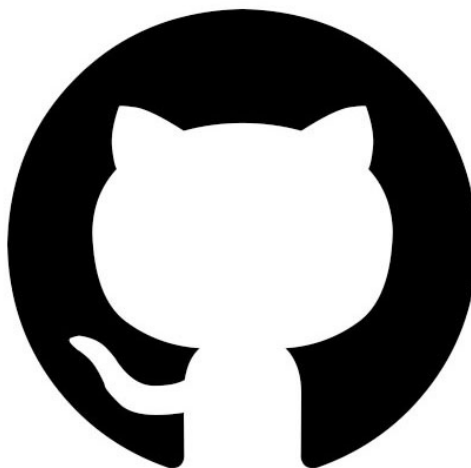
```
https://github.com/github/training-kit
```

Getting Started With Collaboration

We will start by introducing you to Git, GitHub, and the collaboration features we will use throughout the class. Even if you have used GitHub in the past, we hope this information will provide a baseline understanding of how to use it to build better software!

What is GitHub?

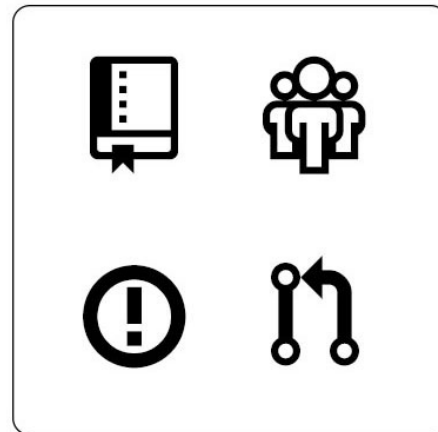
GitHub is a collaboration platform built on top of a distributed version control system called Git.



In addition to being a place to host and share your Git projects, GitHub provides a number of features to help you and your team collaborate more effectively. These features include:

- Issues
- Pull Requests
- Projects
- Organizations and Teams

GitHub



The GitHub Ecosystem

Rather than force you into a "one size fits all" ecosystem, GitHub strives to be the place that brings all of your favorite tools together. For more information on integrations, check out <https://github.com/integrations>.

Explore GitHub Showcases Integrations Trending

Integrations Directory

Use your favorite tools with GitHub.

Powerful integrations that help you and your team build software better, together.

Filters

All integrations

GitHub Enterprise

New & noteworthy

Categories

API management

Chat

Code quality

Code review

Code search

Continuous integration

Travis CI

Test and deploy with confidence

Codeship

Highly customizable Continuous Integration with Docker support

Slack

A messaging app for teams

CircleCI

Automatically build, test, and

Gitter

Helps developers across the

ZenHub

Project Management Inside

You may even find some new, indispensable tools to help with continuous integration, dependency management, code quality and much more.

What is Git?

Git is:

- a distributed version control system or DVCS.
- free and open source.
- designed to handle everything from small to very large projects with speed and efficiency.
- easy to learn and has a tiny footprint with lightning fast performance.

Git features cheap local branching, convenient staging areas, and multiple workflows.

As we begin to discuss Git (and what makes it special) it would be helpful if you could forget everything you know about other version control systems (VCSs) for just a moment. Git stores and thinks about information very differently than other VCSs.

We will learn more about how Git stores your code as we go through this class, but the first thing you will need to understand is how Git works with your content.

Snapshots, not Deltas

One of the first ideas you will need understand is that Git does not store your information as series of changes. Instead Git takes a snapshot of your repository at a given point in time. This snapshot is called a commit.

Optimized for Local Operations

Git is optimized for local operation. When you clone a copy of a repository to your local machine, you receive a copy of the entire repository and its history. This means you can work on the plane, on the train, or anywhere else your adventures find you!

Branches are Lightweight and Cheap

Branches are an essential concept in Git.

When you create a new branch in Git, you are actually just creating a pointer that corresponds to the most recent snapshot in a line of work. Git keeps the snapshots for each branch separate until you explicitly tell it to merge those snapshots into the

main line of work.

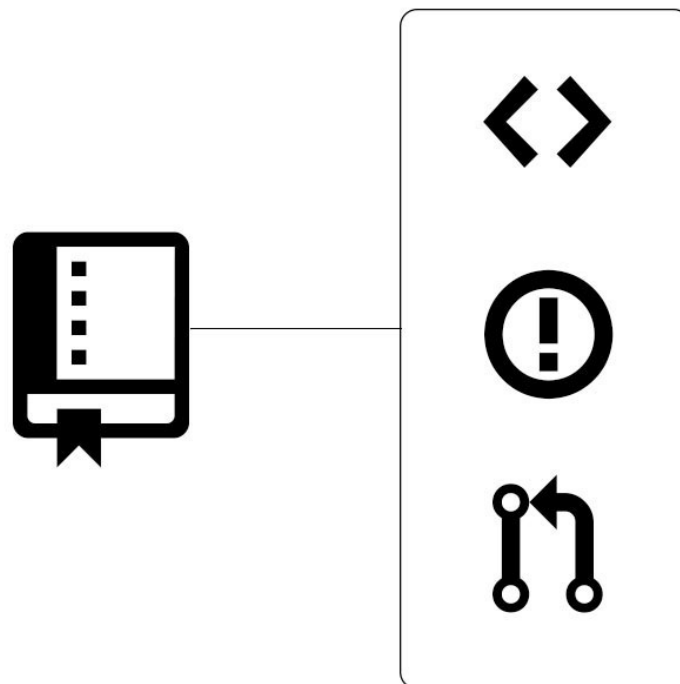
Git is Explicit

Which brings us to our final point for now; Git is very explicit. It does not do anything until you tell it to. No auto-saves or auto-syncing with the remote, Git waits for you to tell it when to take a snapshot and when to send that snapshot to the remote.

Exploring a GitHub Repository

A repository is the most basic element of GitHub. It is easiest to imagine as a project's folder. However, unlike an ordinary folder on your laptop, a GitHub repository offers simple yet powerful tools for collaborating with others.

A repository contains all of the project files (including documentation), and stores each file's revision history. Whether you are just curious or you are a major contributor, knowing your way around a repository is essential!



User Accounts vs. Organization Accounts

There are two account types in GitHub, user accounts and organization accounts. While there are many differences in these account types, one of the more notable differences is how you handle permissions.

User Accounts

When you signed up for GitHub, you were automatically given a user account. Permissions for a user account are simple, you add people as collaborators to specific repositories to give them full read-write access to the project.

Organization Accounts

Organization accounts provide more granular control over repository permissions. In an organization account you create teams of people and then give those teams access to specific repositories. Permissions can be assigned at the team level (e.g, read, write, or admin).

Repository Navigation

Code

The code view is where you will find the files included in the repository. These files may contain the project code, documentation, and other important files. We also call this view the root of the project. Any changes to these files will be tracked via Git version control.

Issues

Issues are used to track bugs and feature requests. Issues can be assigned to specific team members and are designed to encourage discussion and collaboration.

Pull Requests

A Pull Request represents a change, such as adding, modifying, or deleting files, which the author would like to make to the repository. Pull Requests help you write better software by facilitating code review and showing the status of any automated tests.

Projects

Projects allow you to visualize your work with Kanban style boards. Projects can be created at the repository or organization level.

Wiki

Wikis in GitHub can be used to communicate project details, display user documentation, or almost anything your heart desires. And of course, GitHub helps you keep track of the edits to your Wiki!

Pulse

Pulse is your project's dash board. It contains information on the work that has been completed and the work in progress.

Graphs

Graphs provide a more granular view into the repository activity, including who has contributed, when the work is being done, and who has forked the repository.

README.md

The README.md is a special file that we recommend all repositories contain. GitHub looks for this file and helpfully displays it below the repository. The README should explain the project and point readers to helpful information within the project.

CONTRIBUTING.md

The CONTRIBUTING.md is another special file that is used to describe the process for collaborating on the repository. The link to the CONTRIBUTING.md file is shown when a user attempts to create a new issue or pull request.

ISSUE_TEMPLATE.md

The `ISSUE_TEMPLATE.md` (and its twin the pull request template) are used to generate templated starter text for your project issues. Any time someone opens an issue, the content in the template will be pre-populated in the issue body.

Using GitHub Issues

In GitHub, you will use issues to record and discuss ideas, enhancements, tasks, and bugs. Issues make collaboration easier by:

- Replacing email for project discussions, ensuring everyone on the team has the complete story, both now and in the future.
- Allowing you to cross-link to related issues and pull requests.
- Creating a single, comprehensive record of how and why you made certain decisions.
- Allowing you to easily pull the right people into a conversation with @ mentions and team mentions.

Activity: Creating A GitHub Issue

Follow these steps to create an issue in the class repository:

1. Click the *Issues* tab.
2. Click *New Issue*.
3. Type the following in the Subject line: `YOUR-USERNAME Workflow`
4. In the body of the issue, include the text below:

```
YOUR-USERNAME will choose an image, add a caption, and add both to a file.
```

- ```
- [] Create a branch
- [] Edit the file
- [] Commit the changes
- [] Create a Pull Request
- [] Request a Review
- [] Make more changes
- [] Get an approval
- [] Merge the Pull Request
```

## Using Markdown

GitHub uses a syntax called **Markdown** to help you add basic text formatting to Issues, Pull Requests, and files with the `.md` extension.

## Commonly Used Markdown Syntax

### # Header

The `#` indicates a Header. `#` = Header 1, `##` = Header 2, etc.

### \* List item

A single `*` or `-` followed by a space will create a bulleted list.

### \*\*Bold item\*\*

Two asterix `**` on either side of a string will make that text bold.

### - [ ] Checklist

A `-` followed by a space and `[ ]` will create a handy checklist in your issue or pull request.

### @mention

When you `@mention` someone in an issue, they will receive a notification - even if they are not currently subscribed to the issue or watching the repository.

### #975

A `#` followed by the number of an issue or pull request (without a space) in the same repository will create a cross-link.

### :smiley:

Tone is easily lost in written communication. To help, GitHub allows you to drop emoji into your comments. Simply surround the emoji id with `:`.

## Introduction to GitHub Pages

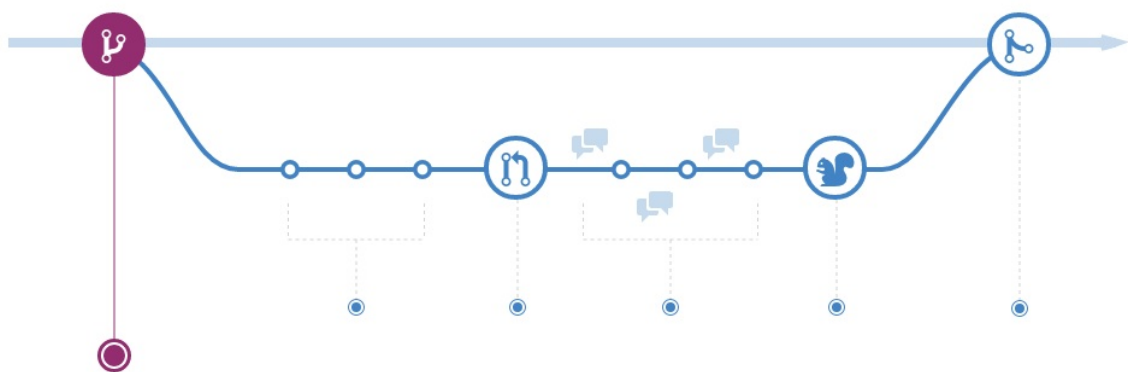
GitHub Pages enable you to host free, static web pages directly from your GitHub repositories. Several of the projects we use in class will use GitHub Pages as the deployment strategy. We will barely scratch the surface in this class, but there are a few things you need to know:

- You can create two types of websites, a user/organization site or a project site. We will be working with project websites.
- For a project site, GitHub will only serve the content on a specific branch. Depending on the settings for your repository, GitHub can serve your site from a `master` or `gh-pages` branch, or a `/docs` folder on the `master` branch.
- The rendered sites for our projects will appear at `githubschool.github.io/repo-name`.

# Branching with Git

The first step in the GitHub Workflow is to create a branch. This will allow us to experiment with new features without accidentally introducing untested changes on our production branch.

## Branching Defined



When you create a branch, you are essentially creating an identical copy of the project at that point in time. This isn't the same as creating a physical copy on disk. In the background, a branch is just a pointer.

Let's learn how you can create a new branch.

## Activity: Creating A Branch with GitHub

Earlier you created an issue about the file you would like to edit. Let's create the branch you will use to edit your file.

Follow these steps to create a new branch in the class repository:

You will need to have collaborator access on the class repository before you can create a branch on GitHub. `{: .warning}`

1. Navigate to *Code* tab of the class repository.
2. Click the *branch dropdown*.
3. Enter the branch name 'github-username-caption'.
4. Press `Enter`.



When you create a new branch on GitHub, you are automatically switched to your branch. Now, any changes you make to the files in the repository will be applied to this new branch.

A word of caution. When you return to the repository or click the top level repository link, notice that GitHub automatically assumes you want to see the items on the default branch. If you want to continue working on a feature branch, you will need to reselect it using the branch dropdown.

## Exploring

Here are some interesting things you can check out later:

- <https://youtu.be/H5GJfcp3p4Q> A GitHub Training Video on branching.

# Local Git Configuration

In this section, we will prepare your local environment to work with Git.

## Checking Your Git Version

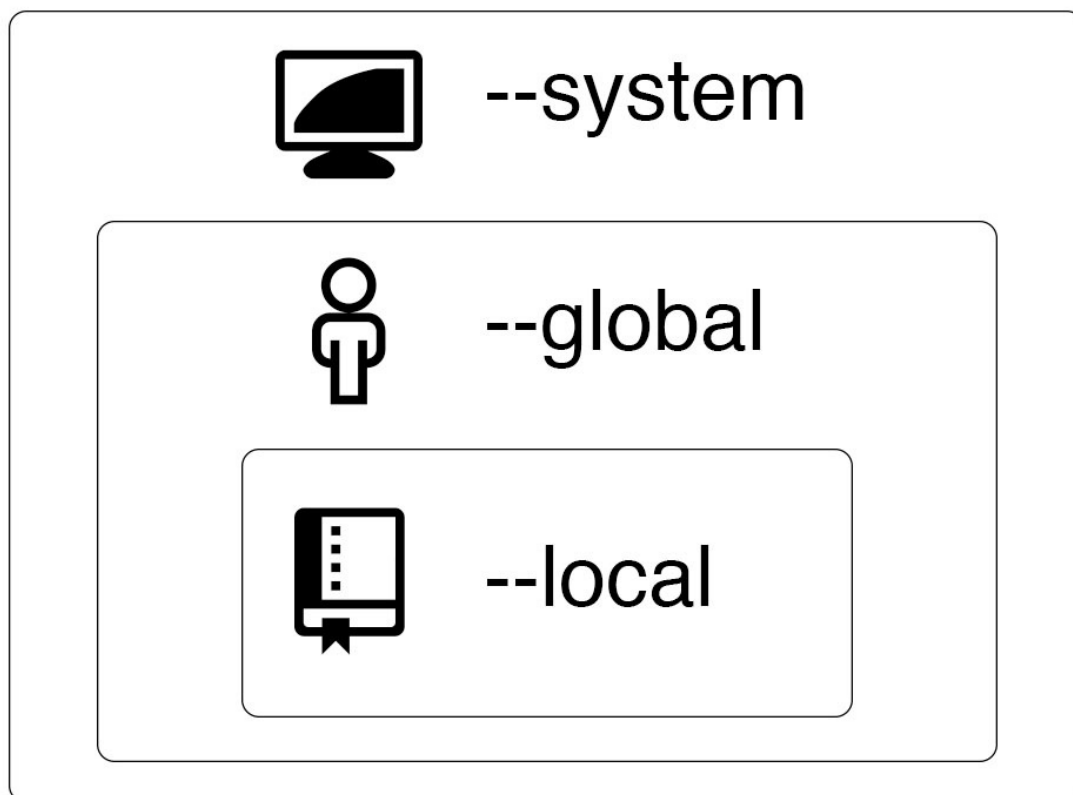
First, let's confirm your [Git Installation](#):

```
$ git --version
$ git version 2.11.0
```

If you do not see a git version listed or this command returns an error, you may need to install Git.

To get the latest version of Git, visit [www.git-scm.com](http://www.git-scm.com).

## Git Configuration Levels



Git allows you to set configuration options at three different levels.

### **--system**

These are system-wide configurations. They apply to all users on this computer.

### **--global**

These are the user level configurations. They only apply to your user account.

### **--local**

These are the repository level configurations. They only apply to the specific repository where they are set.

The default value for `git config` is `--local`.

## Viewing Your Configurations

If you would like to see which config settings have been added automatically, you can type `git config --list`. This will automatically read from each of the three config files and list the settings they contain.

```
$ git config --list
```

You can also narrow the list to a specific configuration level by including it before the list option.

```
$ git config --global --list
```

## Configuring Your User Name and Email

Git uses the config settings for your user name and email address to generate a unique fingerprint for each of the commits you create. You can't create commits without these settings:

```
$ git config --global user.name "First Last"
$ git config --global user.email "you@email.com"
```

## Configuring autocrlf

```
$ //for Windows users
$ git config --global core.autocrlf true
$ //for Mac or Linux users
$ git config --global core.autocrlf input
```

Different systems handle line endings and line breaks differently. If you open a file created on another system and do not have this config option set, git will think you made changes to the file based on the way your system handles this type of file.

Memory Tip: `autocrlf` stands for auto carriage return line feed.